**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING – CYBER SECURITY**
**DAYANANDA SAGAR ACADEMY OF TECHNOLOGY AND MANAGEMENT**
Udayapura, Kanakapura Main Road, Opp. Art of Living,
Bengaluru-82 | 2025–2026



**MINI PROJECT REPORT Continuous Comprehensive Assessment – II (CCA-2)**

ON

# Agentic AI-Based Phishing URL Detection and Analysis System

Submitted by

| Sl. No. | USN | Name |
|---|---|---|
| 1 | 1DT23CY016 | DAKSH |
| 2 | 1DT23CY009 | APURUPA |
| 3 | 1DT23CY010 | ASHA |
| 4 | 1DT23CY035 | PURAB |

Subject: Vulnerability Assessment and Penetration Testing

Faculty Guide:
Shivam Goswami

# DECLARATION

We present this mini project report titled *Agentic AI-Based Phishing URL Detection and Analysis System* as work prepared for the Information and Network Security (BCY404) course. The prototype combines a supervised URL character n-gram Naive Bayes classifier, explainable rules, and a local coordinator. A held-out domain-grouped dataset evaluation is reported with its limitations.

The project is intended for defensive education. Any phishing demonstration with the Social-Engineer Toolkit (SET) must remain inside an isolated lab with explicit authorization and test accounts. The prototype itself does not create phishing pages, send messages, or open submitted URLs.

# ABSTRACT

Phishing URLs use deceptive naming, path and query patterns to direct users toward credential theft or other harmful actions. This mini project implements an offline triage prototype that accepts a URL, parses it locally, applies transparent lexical and structural checks, aggregates weighted indicators into a 0-100 risk score, assigns one of three labels, and explains the result. The workflow is described as an agentic pipeline because a coordinator sequences input validation, selected analysis checks, score aggregation and explanation. No external language model, web request, or VirusTotal lookup is required for routine analysis. The implementation is a supervised Naive Bayes model combined with rule-based auditing. The coordinator validates input, invokes the local model and rule auditor, combines evidence, flags disagreement, and explains results. Inference makes no external AI call or URL request. The model was evaluated on one UCI dataset split, not as a guarantee of live performance. It is designed to demonstrate the architecture and produce interpretable output; its thresholds are illustrative and must be validated on a time-separated, labeled dataset before operational use. False positives, false negatives and changing attacker behavior are material limitations.

**Keywords:** phishing URL, cyber security, explainable analysis, risk scoring, local detection, agent workflow

# TABLE OF CONTENTS


| Section | Title |
|---|---|
| 1 | Introduction |
| 2 | Problem Definition and Objectives |
| 3 | Background and Related Work |
| 4 | System Requirements and Design |
| 5 | Methodology and Algorithm |
| 6 | Implementation |
| 7 | Testing and Evaluation Plan |
| 8 | Results and Discussion |
| 9 | Security, Ethics and Limitations |
| 10 | Conclusion and Future Work |
| References | |

# 1. INTRODUCTION

Phishing is a social-engineering technique that persuades a person to disclose information, approve an action, or visit a harmful destination. A URL can provide useful warning signals before a page is opened: length, host structure, deceptive separators, sensitive terms and transport scheme. These clues are imperfect. HTTPS protects a connection but does not establish that the site is trustworthy, and legitimate sites may use long URLs or words such as “account”.

The supplied project brief asks for an agentic workflow that analyzes URL features, reduces routine dependence on VirusTotal, and explains a verdict. This work combines a trained URL model with an agent-style coordinator. The coordinator sequences local analysis tools, compares outputs, and adds a disagreement warning when needed. It accepts URL text without fetching the destination.

# 2. PROBLEM DEFINITION AND OBJECTIVES

**Problem statement.** Given a URL string, determine whether it should be treated as likely legitimate, suspicious, or likely phishing using locally computed indicators, then provide a score and human-readable reasons. The system must avoid automatically visiting the URL or transmitting it to a third party.

## Objectives

- Parse a supplied URL and normalize a missing scheme for analysis.
- Inspect lexical and structural signals including length, sensitive terms, raw IP hosts, punycode, subdomain depth, @ authority syntax, hyphen density, HTTPS use and query complexity.
- Coordinate checks in a predictable local pipeline and combine evidence into a bounded risk score.
- Return a three-level assessment, contributing reasons and explicit limitations.
- Describe a sound path for measuring precision, recall, false-positive rate and score calibration on labeled data.
- Use SET only as an authorized, isolated educational demonstration; keep offensive activity outside the detector.

# 3. BACKGROUND AND RELATED WORK

URL-based detection is attractive because it can operate before page content is loaded. Research has explored lexical features as inputs to traditional classifiers and character-level representations learned by neural models. URLNet, for example, studies end-to-end URL representation learning for malicious URL detection [1]. Feature selection and dataset construction affect reported results, and test samples should be separated from training data to avoid overly optimistic evaluation.

A rule-based system offers explainable baseline behavior and needs no model download or network service. Its trade-off is limited coverage: fixed terms and weights can be evaded, and legitimate URLs may trigger the same patterns. This project therefore treats the result as triage guidance. Public labeled datasets such as the UCI PhiUSIIL phishing URL dataset can support an evaluation phase [2], subject to license, provenance and data freshness review.

General phishing guidance recommends verifying suspicious requests through a known, independent channel rather than trusting the link [3]. The system reflects that principle by warning that a low score does not prove a URL is safe.

# 4. SYSTEM REQUIREMENTS AND DESIGN

## 4.1 Functional requirements

- Accept one URL string from a command line.
- Return a label, numeric risk score, reasons and safety limitations.
- Support readable terminal output and JSON output.
- Perform all checks locally without opening the destination or requiring VirusTotal.

## 4.2 Non-functional requirements

- Python 3.10+; standard-library inference with the included compressed model.
- Deterministic output for identical input and rule version.
- No network access, page rendering, credential collection or persistence of submissions.
- Show the ML estimate separately from the combined triage score; neither is a safety guarantee.

## 4.3 Architecture

The coordinator validates syntax, invokes the trained Naive Bayes classifier and rule-based auditor, then combines their outputs. With no strong rule flags it weights model 40% and rules 60%; with rule warnings it weights model 70% and rules 30%, while preserving the rule score as a floor. It reports agreement or disagreement and never opens the URL. This is deterministic agent-style ML, not a generative LLM.

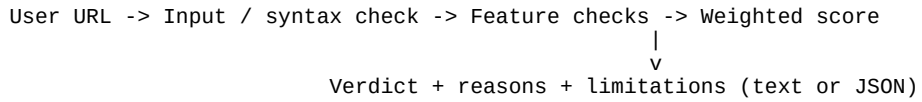



Figure 1. Local analysis pipeline. No browser, DNS lookup, HTTP request, or reputation API is called.

# 5. METHODOLOGY AND ALGORITHM

The model uses character n-grams of lengths 3, 4, and 5 and word tokens from URL paths and queries, plus the scheme and high-risk host structure indicators. It intentionally excludes the host name text so brand words alone do not drive a phishing verdict. Multinomial Naive Bayes learns from labeled examples. The heuristic auditor computes transparent points capped at 100. If there are no strong rule flags, the coordinator weights the model 40%; with rule warnings, it weights the model 70% and keeps the rule score as a floor. Combined-score thresholds below 30, 30-59, and 60+ map to likely legitimate, suspicious, and likely phishing. Scores are triage aids, not proof.

| Signal | Illustrative points | Reason |
|---|---|---|
| URL length > 75 / > 120 | 10 / additional 8 | Long strings can obscure the destination. |
| Sensitive terms | 8-15 per term, cap 25 | Credential, payment or urgency vocabulary. |
| @ in authority | 18 | May visually obscure the effective host. |
| Host is IP literal | 18 | Raw IP destination rather than a familiar domain. |
| At least 5 host labels | 10 | Deep subdomain nesting can disguise a brand-like label. |
| Punycode marker | 15 | May represent an internationalized look-alike domain. |
| At least 3 host hyphens | 8 | One weak lexical signal. |
| Non-HTTPS scheme | 8 | Weak signal only; HTTPS is not a trust guarantee. |
| At least 6 query assignments | 5 | Complex query string can complicate inspection. |

**Algorithm.** Normalize whitespace and parse the input. If parsing fails or no host exists, return a cautious suspicious result with the syntax reason. Otherwise run URL-only checks, collect triggered findings and points, cap the sum at 100, and map the score to a label. If no checks trigger, report that fact and retain a zero score. Always attach the same limitations statement.

The AI component is the supervised Naive Bayes classifier. The agent component routes the URL through syntax validation, ML classification and rule auditing, then arbitrates agreement or disagreement. It is deterministic, local and auditable; it does not use a generative LLM.

# 6. IMPLEMENTATION

The implementation is divided into ml_classifier.py (URL n-gram extraction and inference), detector.py (heuristic checks and agent coordination), and train_model.py (reproducible training and evaluation). The compressed Naive Bayes model ships with the project, so runtime inference uses only the Python standard library.

Training maps UCI label 0 to phishing and label 1 to legitimate. It removes exact duplicate URLs, groups train and test by the dataset Domain column, and uses seed 42. The model uses URL strings and labels only, not website-derived feature columns. At runtime, the submitted URL is parsed but never dereferenced; no DNS, TLS, HTTP, page-content, WHOIS, or reputation lookup is made.

## Example commands

```
python interactive.py
python detector.py "https://example.test/login"
python detector.py "https://example.test/login" --json
```

Output includes the verdict, combined score, model estimate, triggered signals, and coordinator steps. Values depend on the input and the trained model.

# 7. TESTING AND EVALUATION PLAN

The UCI PhiUSIIL dataset contains 235,795 labeled URL/website records. Training uses URL text and labels only; it excludes website-derived feature columns. Exact URL duplicates are removed and domains are held entirely in train or test. With seed 42, 235,370 unique non-conflicting URL records were retained: 188,887 training and 46,483 held-out test examples. Source label 0 (phishing) is positive and label 1 (legitimate) is negative.

The classifier uses binary-presence character n-grams and word tokens from URL paths and queries, plus scheme and selected high-risk host structure indicators; brand/domain text is excluded. Evaluation uses held-out domains and a 0.50 model threshold. This is one reproducible split, not cross-validation or a time-based evaluation.

The dataset is a snapshot; its samples and URL/domain patterns may not represent future attacks. A future study should add time-separated and external datasets, calibrate scores, and review false positives and false negatives.

# 8. RESULTS AND DISCUSSION

On the held-out domain-grouped split of 46,483 URLs, the URL-only classifier achieved 87.22% accuracy, 100.00% precision, 69.55% recall, and 82.04% F1 at a 0.50 model threshold. Confusion counts: 13,568 phishing URLs detected, 5,940 phishing URLs missed, 0 legitimate URLs flagged, and 26,975 legitimate URLs correctly classified.

The result shows that the included model learned useful patterns from this dataset. It must not be read as a real-world guarantee: dataset collection and URL/domain patterns can strongly affect results. The final prototype uses the model together with rules and reports disagreement; both model estimate and combined score are triage signals only.

On the synthetic check URL https://amazon.com, the revised model produced a 2.3% phishing estimate, no rule flags, and a final Likely legitimate verdict with risk score 1/100. This is a local string-analysis example, not a live reputation check. The prototype works without a reputation service and does not fetch submitted URLs. The agent coordinator makes the AI workflow visible by validating, invoking local tools, combining outputs and explaining the result.

# 9. SECURITY, ETHICS AND LIMITATIONS

The code intentionally does not open or contact a submitted URL. For SET learning, use only an isolated lab network, systems owned or explicitly authorized by the institution, synthetic identities and test accounts. Do not target public sites, real users, or production credentials. Keep any demonstration and cleanup under faculty supervision.

## Key limitations

- The model estimate and combined risk score are not calibrated probabilities; weights and thresholds are prototype choices.
- The trained Naive Bayes classifier is supervised ML; the coordinator is deterministic and does not use a generative LLM.
- No domain age, DNS, certificate, redirect, page-content or reputation features are collected.

- Attackers can evade string rules through encoding, Unicode confusables, redirects, shorteners or benign-looking paths.
- A phishing URL may use HTTPS; HTTPS alone neither raises nor lowers confidence in the site owner's intent.
- The system does not replace browser protections, organizational reporting, or human verification.

### Privacy and safe handling

Inputs remain in process memory for a single invocation and are not written to a local history by this implementation. Users should avoid placing sensitive URLs with embedded tokens in shell history or shared logs. An institutional deployment should define retention and access policies before adding logging.

## 10. CONCLUSION AND FUTURE WORK

This mini project delivers an offline supervised URL classifier coordinated with an explainable rule auditor. The agent coordinator validates, routes, combines evidence, detects disagreement and explains results without routine VirusTotal dependence or URL fetching. Evaluation metrics are reported for one domain-grouped split and require cautious interpretation.

Future work should repeat evaluation on time-separated and external datasets, calibrate scores on a validation split, inspect false positives/negatives, and compare model families. Further safe enhancements include Unicode normalization/confusable analysis, public suffix aware domain parsing and optional reputation enrichment behind explicit controls. Any network enrichment must avoid unsafe crawling and document privacy, latency, availability and API limits. A supervised, isolated SET lab demonstration can complement the detector by teaching recognition and response without contacting real targets.

## REFERENCES


[1] S. Le, A. Markopoulou, and M. Faloutsos, "URLNet: Learning a URL Representation with Deep Learning for Malicious URL Detection," arXiv:1802.03162, 2018. https://arxiv.org/abs/1802.03162

[2] A. Prasad and S. Chandra, "PhiUSIIL Phishing URL (Website) Dataset," UCI Machine Learning Repository, 2024. https://doi.org/10.1016/j.cose.2023.103545. Dataset page: https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset

[3] Cybersecurity and Infrastructure Security Agency (CISA), "Recognize and Report Phishing." https://www.cisa.gov/secure-our-world/recognize-and-report-phishing

[4] Mini Project brief supplied for BCY404, "Agentic AI-Based Phishing URL Detection and Analysis System," course handout, 2025-2026.

[5] DSATM, "Mini Project Report Cover Page," supplied template, 2025-2026.

# APPENDIX A - RUNNING THE PROTOTYPE

1. Install Python 3.10 or later. 2. Open a terminal in the project folder. 3. Run the command below with a sample or authorized URL string. The program analyzes text only and does not visit the address.

```
python detector.py "https://sample.example/path"
python detector.py "sample.example/login" --json
```

## A.1 Output fields

| JSON field | Meaning |
|---|---|
| url | Input string as provided. |
| verdict | Likely legitimate, Suspicious, or Likely phishing. |
| risk_score | Combined model/rule triage score from 0 to 100; not a probability. |
| model_phishing_probability | Naive Bayes model estimate from 0 to 1. |
| agent_steps | Coordinator actions and the selected decision branch. |
| findings | Triggered checks, points and human-readable explanations. |
| limitations | Caveats returned with every analysis. |

## A.2 Safe demonstration checklist

- Use synthetic URLs or approved test strings for routine demonstrations.
- Do not click or browse suspicious URLs as part of this command-line analysis.
- If demonstrating SET, use an isolated lab and written authorization from the faculty guide.
- Use synthetic identities and credentials; do not collect real student or public-user data.
- Stop the exercise and notify the guide if the lab system reaches an unapproved network.

The included README summarizes the commands and team details. The program source is `detector.py`.